

System Architecture & Configuration Registry

Workstation Profile: Kubuntu 24.04 LTS

This registry documents the definitive configuration and runtime constraints for this specific system. Stock Kubuntu 24.04 LTS ships with **KDE Plasma 5.27.11 running on X11**. Upgrading to Plasma 6 requires explicitly enabling the **Kubuntu Backports PPA**. The hardware layer—specifically the dual NVIDIA Pascal-architecture GPUs—dictates the entire system posture regarding display protocols, driver pinning, and machine-learning runtime compatibility.

1. Hardware Foundations & System Constraints

1.1 Core Hardware Specifications

- **Motherboard:** ASUS TUF X299 Mark 1
 - *Bus Topology:* `PCIEX16_1` operates at x16 speed. `PCIEX16_3` operates at x8 speed. `PCIEX4_1` shares bandwidth and is forced to x1 speed. Intel RST is active, coupling `M.2_1`, `PCIEX4_2`, and SATA ports 5–8.
- **Processor:** Intel Core i9-10900X (Cascade Lake-X, 10 Cores/20 Threads, 48 PCIe lanes, Dual AVX-512 FMA units, 165W TDP).
- **Memory:** 156.9 GiB DDR4-2933 Quad-Channel RAM. Highly accommodating for large `tmpfs` allocations and aggressive `zram` optimization.
- **Graphics:** 2× NVIDIA TITAN Xp Collectors Edition (Pascal Architecture, GP102 GPU). **Legacy-pinned.**
- **Audio Interface:** Native Instruments Komplete Audio 6 (USB external interface).
- **Network:** Intel I219-V and Intel I211 Gigabit Ethernet controllers (one link active, one administrative down).

1.2 Operating System Baseline

- **Distribution:** Kubuntu 24.04 LTS (Noble Numbat).
- **Stock Desktop Environment:** Plasma 5.27.11, KWin 5.27.11, KDE Frameworks 5, Qt 5.
- **Stock Display Protocol:** X11 (Default). Wayland is experimental on the stock 5.27 base and requires the `plasma-workspace-wayland` package.
- **Kernel:** 6.8 GA baseline (subject to change via Hardware Enablement [HWE] updates).

1.3 NVIDIA Pascal Hardware Dependencies

The Pascal-architecture TITAN Xp GPUs impose hard limits on the machine's software lifecycle:

- **Driver Ceiling:** The NVIDIA **R580 branch** is the final driver line supporting Pascal. The R590 branch drops support entirely. The terminal driver version is **580.105.08**.

- **Kernel Module Constraint:** Open-source NVIDIA kernel modules require a GPU with a Turing or newer architecture (GSP firmware requirement). This system must strictly use the proprietary/closed-source driver packages (`nvidia-driver-580`).
- **CUDA Lifecycle:** CUDA 12.x is the final supported development toolkit. CUDA 13 drops Pascal support. All machine learning virtual environments must target CUDA 12-compatible runtimes.
- **Display Protocol Choice:** While dual Pascal GPUs can run a Wayland session under Plasma 6 via the Backports PPA, X11 remains the configuration with the fewest edge cases for this specific hardware generation.

2. Environment Verification & Discovery Tool

Execute this diagnostic script on the target workstation to determine the exact desktop environment generation and current display state.

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== System Baseline ==="
lsb_release -ds
echo "Kernel: $(uname -r)"
echo "Session Type: ${XDG_SESSION_TYPE:-Unknown}"
echo "Desktop Environment: ${XDG_CURRENT_DESKTOP:-Unknown}"
plasmashell --version 2>/dev/null || echo "Plasma Shell: Not Found"

echo -e "
=== Repository Status ==="
apt-cache policy plasma-desktop 2>/dev/null | head -n 4
ls /etc/apt/sources.list.d/ | grep -i backports || echo "Kubuntu Backports PPA not
enabled."

echo -e "
=== Graphics Driver Status ==="
nvidia-smi --query-gpu=index,name,driver_version,memory.total --format=csv 2>/dev/null ||
echo "NVIDIA SMI failed."
apt-mark showhold | grep -i nvidia || echo "WARNING: NVIDIA driver packages are NOT
pinned."

echo -e "
=== Desktop Portal Backends ==="
/usr/libexec/xdg-desktop-portal --version 2>/dev/null || echo "xdg-desktop-portal not
installed."
ls /usr/share/xdg-desktop-portal/portals/ 2>/dev/null || echo "No desktop portal
configuration found."
```

3. Core System & Security Infrastructure

3.1 Boot & Kernel Parameters

- **Firmware Configurations:** Secure Boot is managed via Machine Owner Keys (`mokutil`). Above-4G Decoding and Resizable BAR are enabled to support multi-GPU memory mapping.
- **Bootloader (`/etc/default/grub`):**

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash nvidia-drm.modeset=1"
```
- **Kernel Modules (`/etc/modprobe.d/`):** Explicitly blacklist `nouveau` to ensure the proprietary NVIDIA stack initializes correctly.

3.2 Service & Process Management

- **System Initialization:** Systemd manages core state transitions up to `graphical.target` .
- **Power & Session Management (`/etc/systemd/logind.conf`):** Explicitly map power, lid, and sleep events to system actions to prevent conflicts with Plasma's user-space management daemons.
- **System Logging:** Journald is configured for persistent disk storage with defined size caps.

3.3 Storage, Filesystems, & Disaster Recovery

- **Volume Structure:** Operating system residing on `/dev/sdd` . Storage managed via GPT partitioning, LVM layers, and standard filesystems (ext4/xfs).
- **Mount Configurations (`/etc/fstab`):** Persistent flags include `noatime` and the scheduling of `fstrim.timer` for solid-state storage.
- **Read-Only Filesystem Resolution Process:** If a volume drops into a read-only state due to safety locks or connection faults, isolate the block layer using the following triage progression before assuming unrecoverable corruption:

```
findmnt
mount | grep ' ro '
lsof +D /mnt/target_path
lsblk -o NAME,RO
blockdev --getro /dev/sdX
```

3.4 Identity, Authentication, & Access Control

- **User Privileges:** System administrators must be members of the `sudo` , `video` , `render` , and `plugdev` system groups.
- **Security Policies:** System-wide rules are enforced via Polkit policies (`/usr/share/polkit-1/rules.d/`) and AppArmor profiles.

4. Package & Repository Strategy

4.1 Apt Package Pinning

To prevent an upgrade path from breaking Pascal compatibility by introducing the R590+ driver branch, explicit version holds are required.

Create a pinning policy file at `/etc/apt/preferences.d/nvidia-580-pin`:

```
Package: nvidia-driver-* nvidia-dkms-* libnvidia-*  
Pin: version 580*  
Pin-Priority: 1001
```

Apply the administrative hold directly to the packages:

```
sudo apt-mark hold nvidia-driver-580 nvidia-dkms-580
```

4.2 Application Runtimes & Python Isolation

- **Sandboxed Packaging:** Flatpak and Snap package types are integrated. System access for Flatpaks is managed via explicit overrides.
- **Python Policy:** This system enforces PEP 668 compliance. Utilizing `--break-system-packages` on the system-wide Python environment is forbidden. All Python development and deployment must occur within virtual environments via tools like `uv` or `venv` inside the user's home directory.

5. Desktop Environment & Graphics Pipeline

5.1 Display Orchestration

- **Stock Base (Plasma 5.27):** Managed by KScreen, storing display configurations in `~/.local/share/kscreen/`. Fractional scaling is implemented globally alongside font DPI workarounds on X11.
- **Backports Branch (Plasma 6.x):** Introduces native per-output fractional scaling and experimental HDR/Wide Gamut mapping when running on the Wayland display protocol.

5.2 Window Management (KWin)

- **Configuration:** Behavior rules are written directly to `kwinrc`.
- **Window Rules:** Window positioning, layout behavior, and task-switching logic are written to `kwinrulesrc` and reloaded programmatically:

```
qdbus org.kde.KWin /KWin reconfigure
```

6. XDG Desktop Portals & Middleware Integration

The desktop portal layer (`xdg-desktop-portal`) operates as the critical translation matrix between sandboxed user applications and the display compositor. This is a vital layer when running under Wayland.

6.1 Backend Verification

The primary orchestration engine must route directly through the native KDE implementation (`xdg-desktop-portal-kde`). The GTK backend (`xdg-desktop-portal-gtk`) serves strictly as a fallback.

6.2 Key Integrated Subsystems

- **Screen Casting & Remote Management:** Managed via PipeWire streams. Screencasting engines (e.g., OBS Studio, Spectacle) interact directly with this backend. Ensure that screen capture functionalities resolve correctly alongside the legacy NVIDIA driver.
- **Input Capture Interface:** Required for running remote input translation utilities like Input Leap or virtual KVM solutions under Wayland.
- **File Chooser Operations:** Ensures that isolated flatpak and snap runtimes default back to native KDE file picker dialogs instead of falling back to raw GTK menus.

7. Device, Network, & User Experience Profiles

7.1 Peripherals & Input Management

- **Audio Architecture:** Managed via PipeWire graph scheduling. The external Native Instruments Komplete Audio 6 interface is assigned static priority for system output and default capture profiles.
- **Input Hardware:** Core settings for keyboards, pointing devices, and digital drawing tablets are mapped directly through the respective system setting modules.

7.2 Connectivity Configuration

- **Network Mapping:** Core profiles are evaluated by NetworkManager and mapped against system-assigned interface nodes (e.g., `i219-V` / `i211`).
- **Remote Workstation Access:** VNC, KRfb, or KRdp services handle incoming session initialization requests.

7.3 Data Indexing

- **Search Engine:** Baloo handles local file metadata scanning. If disk utilization or I/O pressure becomes anomalous, state parameters can be modified using the following tool:

```
balooctl status
```

8. Custom Desktop Launcher Integration

To integrate local network targets or localized development sites directly into application menus, KRunner, and system application docks, conform explicitly to the freedesktop standard launcher structure.

8.1 Launcher Configuration File

Create the customized entry file at `~/.local/share/applications/local-webapp.desktop`:

```
[Desktop Entry]
Type=Application
Name=Local Web Workspace
Comment=Development and Dashboard Interface
Exec=chromium-browser --app=http://localhost:8080/ --user-data-dir=%h/.local/share/webapps/workspace
Icon=network-server
Categories=Development;Network;
Keywords=local;dashboard;development;tools;
StartupWMClass=localhost__8080
StartupNotify=true
```

8.2 Database Synchronization

After saving or altering any `.desktop` configuration template, rebuild the system launcher tree:

```
update-desktop-database ~/.local/share/applications
```

9. Configuration Automation & Disaster Recovery

9.1 Desktop Parameter Automation

- **Low-Level Direct Manipulation:** Configuration attributes can be modified at the CLI without accessing system settings panel dialogs:

```
kwriteconfig6 --file kdeglobals --group Colors:Window --key BackgroundNormal "45,45,45"
```

- **Declarative Infrastructure:** For full system replication, declarative structures can be implemented using specialized orchestration playbooks (e.g., Ansible's `community.general.kdeconfig` or declarative package state files).

9.2 Complete Backup Mapping

To fully recreate this precise environment state from bare-metal installations, track, version-control, and archive the following configuration structures:

```
/etc/apt/preferences.d/nvidia-580-pin      # Critical driver package locks
/etc/modprobe.d/                          # Blacklists and boot parameters
```

```

/etc/sddm.conf.d/           # Display manager settings
~/.config/*rc               # Individual application preferences
~/.config/kdeglobals        # Global appearance properties
~/.config/kglobalshortcutsrc # Global keybind mappings
~/.config/kwinrc            # Window manager configurations
~/.config/kwinrulesrc       # Targeted application window rules
~/.config/plasma-workspace/  # Workspace session setup scripts
~/.local/share/applications/ # Custom application launcher hooks
~/.local/share/kscreen/     # Hardware display configuration arrays

```

10. Summary Configuration Index

Profile Parameter	System Implementation Style	Dependency Vector
System Boot & Hardware	Static configurations	Physical host system attributes
Kernel Modules & Power	System-wide initialization settings	Physical hardware configuration
Storage & Filesystems	Native Linux disk structure	Physical hardware configuration
Apt Package Tree	Mandatory target version holds	Driver constraints imposed by Pascal GPUs
Graphics & Compositing	Protocol choices shift dynamically	Determined by chosen desktop platform track
Desktop Integrations	Native portal coordination	Vital under Wayland protocols
Window Layout Rules	Dynamic KWin rules engines	Defined by user desktop environment track
Automation Strategy	Declarative state tools	Determined by choice of config tools